The design of lock-based concurrent data structures is all about ensuring that the right mutex is locked when accessing the data and ensuring that the lock is held for a minimum amount of time. This is hard enough when there's just one mutex protecting a data structure. You need to ensure that data can't be accessed outside the protection of the mutex lock and that there are no race conditions inherent in the interface, as you saw in chapter 3. If you use separate mutexes to protect separate parts of the data structure, these issues are compounded, and there's now also the possibility of deadlock if the operations on the data structure require more than one mutex to be locked. You therefore need to consider the design of a data structure with multiple mutexes even more carefully than the design of a data structure with a single mutex.

In this section you'll apply the guidelines from section 6.1.1 to the design of several simple data structures, using mutexes and locks to protect the data. In each case you'll seek out the opportunities for enabling greater concurrency while ensuring that the data structure remains thread-safe.

Let's start by looking at the stack implementation from chapter 3; it's one of the simplest data structures around, and it uses only a single mutex. Is it really thread-safe? How does it fare from the point of view of achieving true concurrency?

6.2.1. A thread-safe stack using locks

The thread-safe stack from chapter 3 is reproduced in the following listing. The intent is to write a thread-safe data structure akin to `std::stack<>`, which supports pushing data items onto the stack and popping them off again.

Listing 6.1. A class definition for a thread-safe stack